

Assignment 1

Biprarshi Biswas

August 17, 2026

1 Introduction

This document is the supplementary report for Assignment 1. A frame based data transmission system with error detection. The system establishes a TCP socket connection between a sender and a receiver process. It builds fixed size frames from an input file, computes a frame check sequence using either a 16 bit checksum or one of four CRC variants, and verifies each frame at the receiver. A separate error injection module tests all error detection schemes against several classes of corruption to confirm their theoretical detection guarantees. Finally, a performance module measures the computational throughput of each scheme.

The report covers four parts. Section 2 describes the system architecture and file layout. Section 3 explains the mathematics behind checksum and CRC. Section 4 covers the error injection module and its results. Finally, Section 5 compares computational performance of all error detection schemes discussed.

2 Architecture

This section provides an overview of the technical architecture of the implementation. We document a tour of various modules and their respective functions.

2.1 File Structure

Build artifacts and log files are omitted below for readability.

```
.
|-- lab1/
|   |-- data.txt
|   |-- error.c, error.h
|   |-- frame.h
|   |-- scheme.c, scheme.h
|   |-- sender.c, receiver.c
|   |-- Makefile
|   |-- README.md
|   |-- error_inject/
|   |   |-- bit_flip.c, bit_flip.h
|   |   |-- targeted_inject.c, targeted_inject.h
|   |   '-- eval_main.c
|   |-- perf_analysis/
|   |   |-- perf_analyzer.c, perf_analyzer.h
|   |   '-- perf_main.c
|   |-- tests/
|   |   |-- error_test.c, checksum_test.c, crc_test.c
|   |   |-- scheme_test.c, boundary_test.c
|   |   '-- (additional exploratory tests)
'-- utils/
    |-- utils.c
    '-- utils.h
```

2.2 Network Utilities

`utils.c/h` provides a scheme agnostic TCP layer used by both sender and receiver across assignment. Connection setup functions (`listen`, `accept`, `connect`) accompanied with transfer functions that loop until a full buffer is sent or received through `read/write` calls.

2.3 Frame Structure

Every frame is 64 bytes, split into three fixed regions.

[HEADER 16B] [PAYLOAD 44B] [TRAILER 4B]

Field	Size	Purpose
Source address	6 B	Sender identifier
Destination address	6 B	Receiver identifier
Frame length	2 B	Valid payload bytes in this frame (≤ 44)
Frame sequence	2 B	Ordering
Payload	44 B	User data payload
FCS (trailer)	4 B	Checksum or CRC result, zero extended

Table 1: Frame field layout

The 64 byte size matches the Ethernet 802.3 minimum frame size. The FCS is computed solely over the header and payload.

2.4 FCS Compute and Verify

The sender fills the header, converts host byte order to network byte order and finally computes the FCS over the header and payload region and stores it in the trailer. The receiver recomputes the error detection function over the same region and compares against the received FCS. Section 3 covers the mathematics of all error schemes in detail.

2.5 Error Injection Module

A dedicated module (`error_inject/`) introduces controlled corruption into a frame after its FCS has been computed, then checks whether checksum and CRC can detect the corruption. It includes both randomized and deliberate error injectors that target vulnerabilities in error detection. Full detail and results are documented in Section 4.

2.6 Performance Analysis Module

A separate module (`perf_analysis/`) measures sustained computational throughput of each scheme under a fixed time budget. Full detail and results are in Section 5.

2.7 Test Suite

Implementation of project followed test-driven-development convention, adding tests as milestones reached covering both normal and boundary behavior.

Test File	Coverage
<code>error_test.c</code>	Integration: checksum + all CRC variants
<code>checksum_test.c</code>	10 tests: edge cases, patterns, bit flips
<code>crc_test.c</code>	8 tests: empty data, standard vectors, uniqueness
<code>scheme_test.c</code>	8 tests: dispatcher parsing, independence, tampering
<code>boundary_test.c</code>	10 tests: exhaustive Hamming distance checks

Table 2: Test coverage by file

3 Error Detection Schemes

This section documents the mathematical reasoning behind Checksum and CRC error detection algorithms.

3.1 16-bit Checksum

The message is split into 16 bit words $w_1, \dots, w_n$ and are added using one's complement addition. Here, ordinary binary addition is carried out, except any carry out on bit 15 is added back into bit 0. This is equivalent to computing

$$S = \left(\sum_{i=1}^n w_i \right) \bmod (2^{16} - 1)$$

The checksum is the bitwise complement of this sum, $C = \bar{S}$. Complementing a value and adding it back to the original yields all ones: $S + C = 2^{16} - 1$.

For verification, the receiver adds every received word, including the checksum field itself, using the same one's complement addition. If no error occurred, this sum folds to exactly $2^{16} - 1$ (all ones, `0xFFFF`). Any other result means the frame is corrupt and subsequently rejected.

The final complement step exists specifically to remove a known blind spot. A message and checksum field that are both zeroed either deliberately or through a fault would sum to zero and pass silently, since zero is a valid sum for a naive scheme. With the complement, an all zero message has checksum `0xFFFF` and this failure mode is addressed.

3.2 Cyclic Redundancy Check

A message of n bits is treated as a polynomial $M(x)$ over $\text{GF}(2)$, where each bit acts as a coefficient. A generator polynomial $G(x)$ of degree r is fixed in advance. The sender computes the remainder $R(x)$ of $M(x) \cdot x^r$ divided by $G(x)$, using polynomial division over $\text{GF}(2)$. The transmitted codeword is

$$T(x) = M(x) \cdot x^r + R(x)$$

which is always exactly divisible by $G(x)$.

The receiver divides the received bits by $G(x)$. A remainder of zero means no error was detected and any other results means the frame is corrupt and subsequently rejected. In practice this division is computed bit by bit with a shift register of width r : each incoming bit shifts into the register, and whenever the bit shifted out is a one, the register is XORed with $G(x)$.

Scheme	Generator Polynomial	Hex
CRC-8	$x^8 + x^7 + x^6 + x^4 + x^2 + 1$	<code>0xD5</code>
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x + 1$	<code>0x233</code>
CRC-16	$x^{16} + x^{15} + x^2 + 1$	<code>0x8005</code>
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	<code>0x04C11DB7</code>

Table 3: Generator polynomials used

The CRC error detection schemes yields three guarantees. Any single bit error is always detected, since $G(x)$ has more than one term. Any error with an odd number of flipped bits is detected whenever $G(x)$ has $(x + 1)$

as a factor, which holds for all four polynomials above. Any burst error confined to r or fewer consecutive bits is always detected, since such a burst cannot be a multiple of a degree r polynomial. Beyond that length, a fraction $1 - 2^{-r}$ of random error patterns are still caught. However, an error is invisible when its error polynomial $E(x)$ is itself a multiple of $G(x)$, since $T(x)$ and $T(x) + E(x)$ then leave the same remainder

4 Error Injection Module

This section documents the various cases of corruption and compares how each error detection scheme handles them.

4.1 Overview

The module has two layers in its implementation. `bit_flip.c` provides the primitive of flipping one specific bit in a buffer by XOR, addressed by a global bit index. `targeted_inject.c` builds two vulnerabilities on top of this primitive, each aimed at one scheme's specific blind spot. `eval_main.c` drives all test cases across trials, logs results to console and to `results.csv`.

4.2 Test Cases

- 0) **All-zero blind spot (checksum)** - where data and checksum fields are both zeroed, checking whether the one's complement design avoids the classic naive-sum failure mode.
- 1) **Compensating error (checksum)** - where the same bit column flipped in two 16-bit words, targeting checksum's blind spot for compensating errors that cancel each other in the sum.
- 2) **Generator multiple (CRC)** - where the generator polynomial's own bit pattern is XORed into the frame.
- 3) **Burst length at r vs $r + 1$ (CRC)** - which tests the burst errors of length $r + 1$ which are caught with probability $1 - 2^{-r}$.

4.3 Results

Case	Expected	Observed
0: All-zero blind spot	Undetected	Detected

Table 4: Case 0 result (single deterministic construction)

CRC Degree	Checksum Caught	CRC Caught
8	0/10000 (0%)	10000/10000 (100%)
10	0/10000 (0%)	10000/10000 (100%)
16	0/10000 (0%)	10000/10000 (100%)
32	0/10000 (0%)	10000/10000 (100%)

Table 5: Case 1: compensating error, 10000 trials per degree

The results above confirm our hypothesis. Checksum detection is exactly 0% at every degree, while CRC detects every case, since polynomial division treats the two flips as independent bit positions rather than as a linear sum.

CRC Degree	Checksum Caught	CRC Caught
8	10000/10000 (100%)	0/10000 (0%)
10	10000/10000 (100%)	0/10000 (0%)
16	10000/10000 (100%)	0/10000 (0%)
32	10000/10000 (100%)	0/10000 (0%)

Table 6: Case 2: generator multiple error, 10000 trials per degree

The injected pattern is exactly the generator polynomial’s bits, so the corrupted codeword remains a multiple of $G(x)$ and CRC’s remainder stays zero regardless of the underlying corruption. The same bit pattern changes the checksum’s arithmetic sum in essentially every case, so checksum detects it every time. Results confirm out hypothesis over all four degrees.

CRC Degree	Burst= r	Burst= $r+1$ observed	Burst= $r+1$ theoretical
8	10000/10000	10000/10000 (100%)	99.609%
10	10000/10000	10000/10000 (100%)	99.902%
16	10000/10000	10000/10000 (100%)	99.998%
32	10000/10000	10000/10000 (100%)	100.000%

Table 7: Case 3: burst error boundary, 10000 trials per degree

Burst length r is detected without exception at every degree, which matches the guarantee directly. Burst length $r + 1$ shows 100% detection at every degree as well, including CRC-8, where theory predicts a minute miss rate.

5 Performance Analysis

This section formalizes the performance metrics and compares throughput accros error detection schemes.

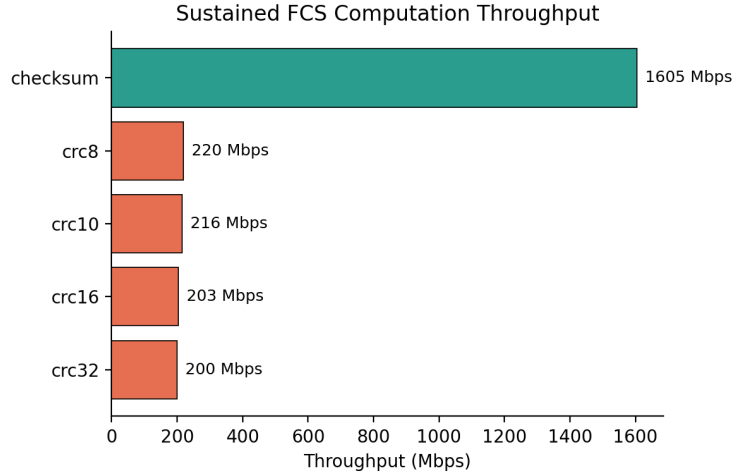
5.1 Methodology

Each scheme is run in a loop over randomly generated 64 byte frames for a fixed 10 second window, timed with `clock_gettime(CLOCK_MONOTONIC)`. The loop measures only the FCS computation itself, with no socket I/O or frame packing included, so the result reflects the isolated cost of the error detection algorithm. Throughput in bits per second is the primary metric.

5.2 Results

Scheme	Throughput (Mbps)	Frames/sec	Speedup vs CRC-32
checksum	1605	3,135,052	8.0×
crc8	220	430,355	1.1×
crc10	216	422,563	1.08×
crc16	203	396,787	1.01×
crc32	200	391,562	1.0×

Table 8: Sustained throughput per scheme

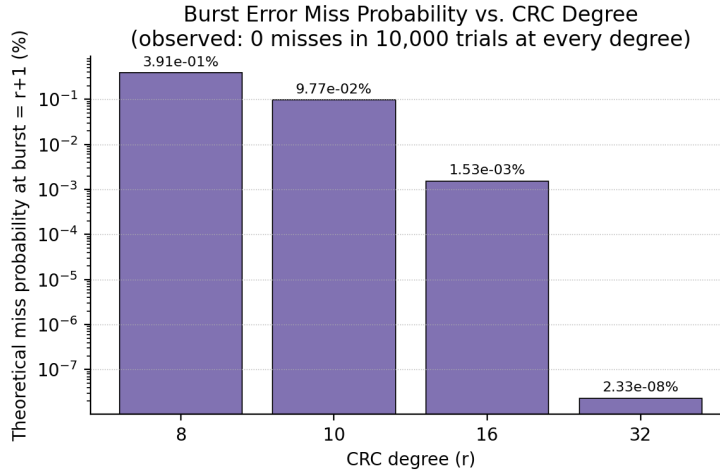


5.3 Discussion

Checksum is roughly eight times faster than any CRC variant. Checksum computes one addition per 16 bit word, while CRC performs a shift and conditional XOR for every bit of input.

CRC-8 through CRC-32 cluster within about 10% of each other despite an eight to sixteen fold difference in output width. This is expected under a bit serial implementation, where cost scales with the number of input bits processed, not the register width. Therefore, widening the polynomial from 8 to 32 bits adds fixed extra computation rather than multiplying the total cost.

Neither scheme is a practical bottleneck at typical network speeds.



The theoretical miss probability at burst length $r + 1$ falls by roughly two orders of magnitude for every doubling of CRC degree. This is the reasoning behind choosing a wider polynomial when burst errors longer than the guarantee threshold are a real possibility, at the cost of the throughput difference discussed above.